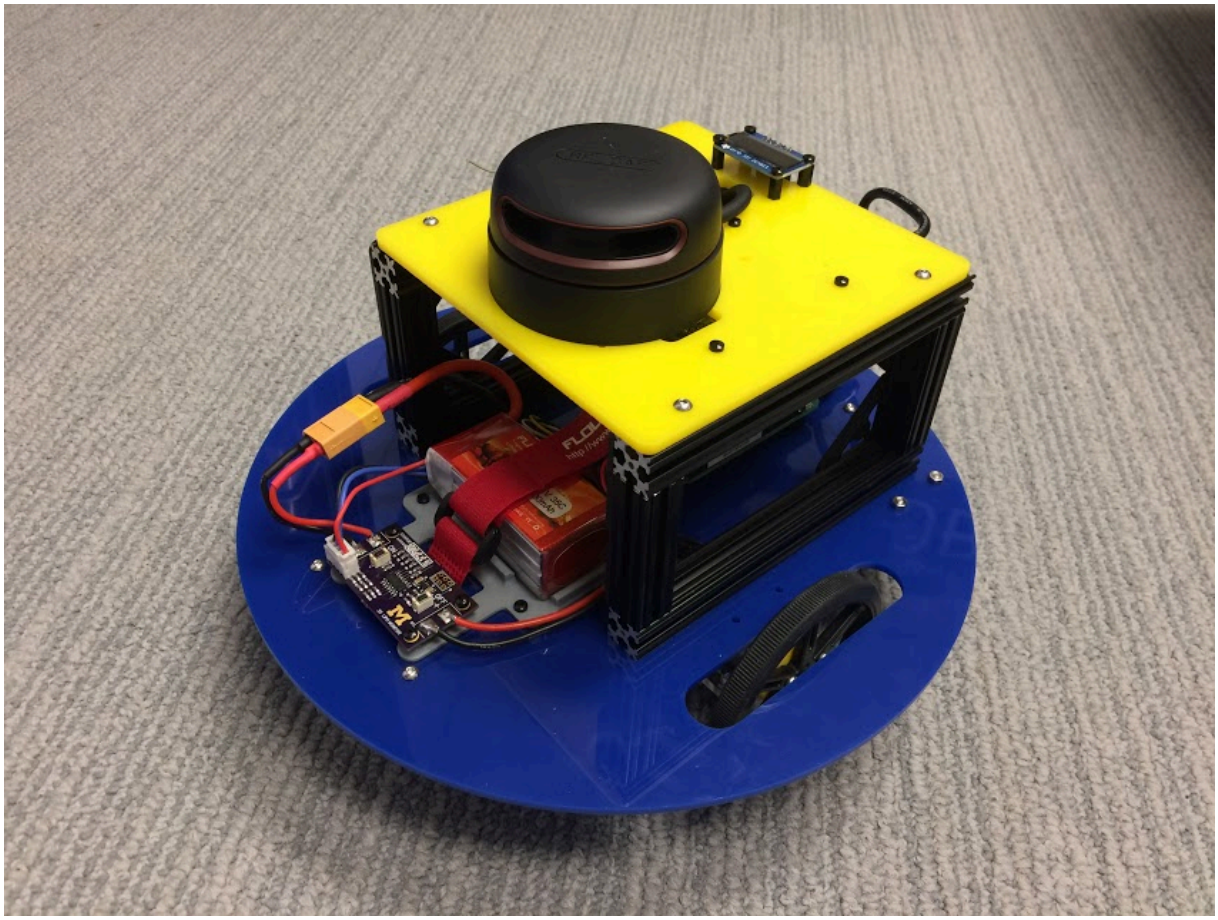
BotLab



Overview

ACTING

- Planar kinematics of a differential-drive ground robot
- Motion models with uncertainty

PERCEPTION

- Quadrature Encoders
- MEMS Inertial Measurement Unit
- 2D LIDAR Rangefinders

REASONING

- Monte Carlo Localization
- Simultaneous Localization and Mapping
- A* search
- Path planning

Overview

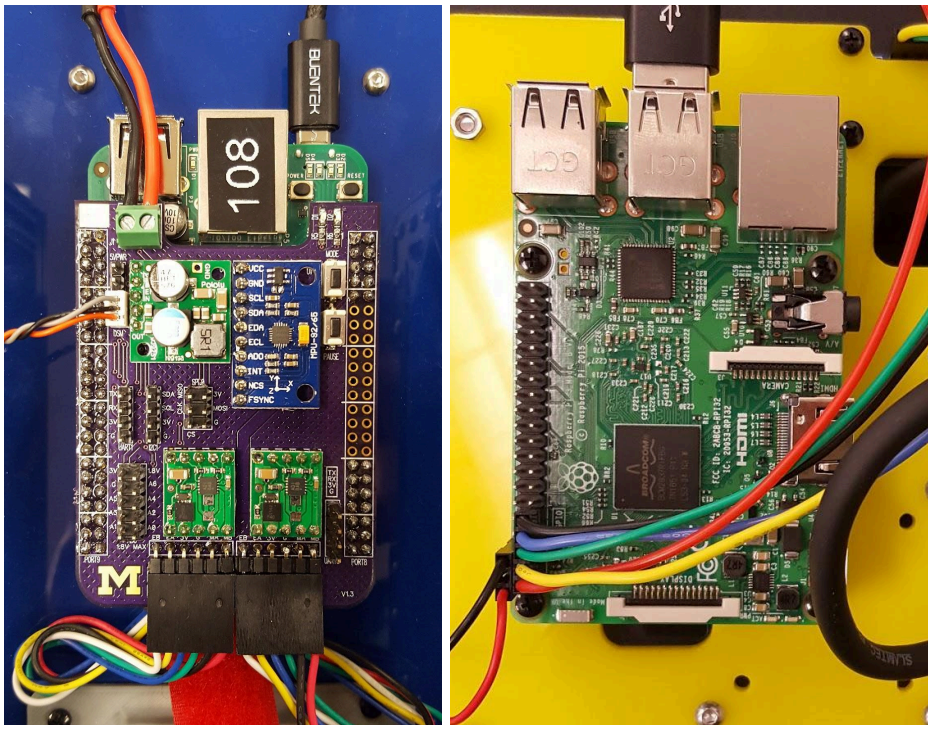
In this lab, you will be working with code to run on the MBot. Each MBot moves with differential drive (2 parallel wheels with a rear and front caster to prevent tipping over) and is equipped with magnetic wheel encoders, a 2D Lidar, and a MEMS 3-axis IMU. The MBot has a WiFi connection for communicating with its onboard Raspberry Pi 3 (RPi3) computer module. The low level motor control is handled on an onboard Beaglebone Green (BBG) with a Mobile Robotics Cape which can be accessed through the RPi3.

Part 1:
You will implement three separate features for the MBot, a simple simultaneous localization and mapping (SLAM) system using the 2D lidar, wheel encoders, and IMU.

Part 2:
You will implement path planning and navigation algorithms for a virtual M-Bot.

Getting Started:

The MBot utilizes two development boards for its operation: A Raspberry Pi 3 performs SLAM calculations, path planning, and collects 2D Lidar Measurements. A Beaglebone Green with a Mobile Robotics Cape interfaces with the wheel encoders, IMU, and motors and handles motor commands, velocity control, and odometry.



Beaglebone Green with cape (Left) and the Raspberry Pi 3 (Right)

botgui

The botlab repository also contains a Graphical User Interface for displaying the status, and sensor measurements of the MBot as it runs its exploration tasks. This can be set up by cloning the botlab repository onto your computer and building the GUI

```
On a laptop:
$ git clone https://gitlab.eecs.umich.edu/YOUR_ACCOUNT/botlab-w20.git
$ make laptop-only
```

Testing Template Code

In a terminal:

1. Source the paths in setenv.sh: `. setenv.sh`
2. Change to the log file directory `cd data`
3. run `lcm-logplayer-gui <any log file>`

In another terminal:

4. Source the paths in setenv.sh: `. setenv.sh`
5. Change to the bin directory `cd bin`
6. run `./botgui`

You should see green rays representing lidar scans printed to the botgui. You can also get an idea of all lcm messages are being transmitted by running lcm-spy in another terminal on your laptop:

In another terminal:

7. Source the paths in setenv.sh: `. setenv.sh`
8. run `$ lcm-spy`

Software

MBot programs run in three places: the Beaglebone Green (mobilebot-w20), Raspberry Pi (botlab-w20 and botlab-startup-scripts [already on RPi3]), and your laptop (botlab-w20). But because all messages are passed using LCN, the software

`src/slam/*` - Modify the files in this directory to implement your slam program

`src/planning/*` - Modify the files in this directory to implement your path planning program

`src/apps/*` - contains the source code for the botgui interface. This builds to the `bin/botgui` executable. You do not need to modify this code for it to work, but if you want to add features to your GUI, this is where to do it.

LCM Communication

The MBot utilizes the Lightweight Communications and Marshalling (LCM) protocol for transmitting data between programs:

https://lcm-proj.github.io/

LCM channels are published and subscribed to by various programs and data is sent through each channel in the form of an lcm datatype.

Listed here are the lcm datatypes that the MBot uses, the Channel used, and the program that broadcasts that datatype. The destinations of these datatypes are listed (although some of these datatypes have multiple destinations). The destination code MC stands for motion_controller.

Type	Channel	Src.	Dest.
exploration_status_t	EXPLORATION_STATUS	exploration	botgui
mbot_motor_command_t	MBOT_MOTOR_COMMAND	motion_controller	mobilebot
odometry_t	ODOMETRY	mobilebot	MC
pose_xyt_t	SLAM_POSE	slam	MC
robot_path_t	CONTROLLER_PATH	exploration	MC
timestamp_t	MBOT_TIMESYNC	timesync	mobilebot
rplidar_laser_t	RPLIDAR_LASER	rplidar_driver	slam
oled_message_t	OLED_MESSAGE	various programs	stats.py
occupancy_grid_t	SLAM_MAP	slam	botgui
particles_t	SLAM_PARTICLES	slam	botgui

LCM Communications Information

LCM uses UDP multicast, part of the Internet Protocol (IP) to transmit messages over the network. IP packets have a specific TTL (time to live) value associated with each packet. Packets with a TTL of 0 stay on the host. Packets with a TTL of 1 are restricted to the same subnet. Each router that sees a packet will check its TTL and decrement it before sending it out to the next network layer, so packets do not live indefinitely on the global internet. When running the programs we tell LCM which multicast address to use, and what the TTL of our LCM messages is. In the case of mobilebot, this is defined in mb_defs.h. In this case we have

```
#define LCM_ADDRESS "udpm://239.255.76.67:7667?ttl=2"
```

LCM communication with a source and destination program running on the same machine (e.g. exploration --> motion_controller) will happen automatically as long as an interface on the machine is configured to use multicast. If no interface is configured this way, LCM will print an error message and the program will exit (possibly with a SEGFAULT).

LCM messages are played back from a log file with All LCM messages can be viewed with LCM Spy. It will show all LCM communications but will not be able to decode messages until you source the setenv.sh script.

```
. setenv.sh
```

OLED Debug Display

An example for writing debug messages to the OLED display is given in botlab-w20/src/mbot/oled_example.cpp. The oled display datatype contains two lines of debug output that you can set to whatever helps with debugging. You can also modify what each of the four lines of the OLED prints out. This can be done by modifying the `lines` array in `stats.py` (note: you should leave in the line displaying the IP address as it will make connecting to the Pi difficult if you do not have this information).

PART 1: SLAM

During the SLAM part of the lab , you will build an increasingly sophisticated algorithm for mapping and localizing in the environment. You will begin by constructing an occupancy grid using known poses. Following that, you'll implement Monte Carlo Localization in a known map. Finally, you will put each of these pieces together to create a full SLAM system. Much of this can be initiated using only logs and when you are ready to use the actual robot, request the upper chassis to mount on your robot with a LIDAR and a RPi.

LCM Logs

For this phase, we have provided two LCM logs containing sensor data from the MBot along with ground-truth poses as estimated by the staff's SLAM implementation.

- `data/convex_10mx10m_5cm.log` : convex environment, where all walls are always visible and the robot remains stationary (use for initial testing of algorithms).
- `data/drive_square_10mx10m_5cm.log` : a convex environment while driving a square
- `data/obstacle_10mx10m_5cm.log`: driving a circuit in an environment with several obstacles

To play back these recorded LCM sessions on a laptop (with Java), you can use the `lcm-logplayer-gui`

```
:~$ lcm-logplayer-gui data/convex_10mx10m_5cm.log
```

Note: you can also run the command line version, lcm-log-player, if the system does not have Java. In the GUI version you can turn off LCM channels with a checkbox. The same functionality is available in the command line version, check the help with

```
:~$ lcm-logplayer --help
```

Similarly, to record your own lcm logs, for testing or otherwise, you can use the `lcm-logger`

```
:~$ lcm-logger -c SLAM_POSE_CHANNEL my_lcm_log.log
```

This example would store data from the OPTITRACK_CHANNEL in the file `my_lcm_log.log` . With no channels specified, all channels will be recorded over the interval.

Accounting for Scan Speed

The laser rangefinder beam rotates in 360 degrees at a sufficiently slow rate such that the robot may move a non-negligible distance in that time. Therefore, you will need to estimate the actual pose of the robot when each beam was sent in order to determine the cell in which the beam terminated. To do so, you must interpolate between the robot pose estimate of the current scan

and the scan immediately before. Remember that the pose estimate of the previous SLAM update occurred immediately before the current scan started. Likewise, the pose estimate of the current SLAM update will occur when the final beam of the current scan is measured. We have provided code to handle this interpolation for you, which is located in `src/slam/moving_laser_scan.hpp`. You only need to implement the poses to use for the interpolation.

1.1 Mapping - Occupancy Grid:

Implement the occupancy grid mapping algorithm in the Mapping class in `slam/mapping.cpp/.hpp`.

An occupancy grid describes the space around the robot via the probability that each cell of the grid is occupied or free. We will use a grid with at most 10 cm cells, whose log-odds values are integers in the range [-127,127]. To perform the ray casting on your occupancy grid you might want to implement Breshenham's line algorithm as described in class lecture.

For this Task, use the ground-truth poses in the provided log files to construct occupancy grid maps of each one. Using the poses from the log file is handled by specifying the --mapping-only command-line option when you run the slam program.

DELIVERABLES:

- Run ./slam in --mapping-only mode, form a map in botgui while recording a log and provide a png of it for:
  - convex_10mx10mx5cm_offcenter.log
  - drive_square_10mx10m_5cm.log
  - Run1_maze_lowres/Maze_lowres_6_6_2.log (from a clone of botlab-data repo)
- Write a paragraph or two description including the following:
  - Provide the values of the incremental log odds you're using for a free & occupied cells
  - Describe the method used determine which cells to update
  - Comment on mapping behaviour

1.2 Monte Carlo Localization:

As discussed in lecture, Monte Carlo Localization (MCL) is a particle-filter-based localization algorithm. Implementation of MCL requires three key components: an action model to predict the robot's pose, a sensor model to calculate the likelihood of a pose given a sensor measurement, and various functions for particle filtering including drawing samples, normalizing particle weights, and finding the weighted mean pose of the particles. For Tasks 1.2.1-1.2.3, you will implement MCL using odometry, laser scans, and an occupancy grid map.

In these tasks, you'll run the slam program in localization-only mode using a saved map. Use the ground-truth maps provided with the sensor logs. You can run slam using:
`./slam --localization-only <filename>.`

To test the action model only, you can run in action-only mode with localization-only turned on:
`./slam --localization-only <filename> --action-only`

1.2.1 MCL - Action Model:

Implement an action (or motion) model using odometry or wheel encoder data. The skeleton of the action model can be found in `slam/action_model.h/.cpp`.

Refer to Chapter 5 of Probabilistic Robotics for a discussion of common action models. You can base your implementation on the pseudo-code in this chapter. There are two action models that are discussed in detail, The Velocity Model (Sec. 5.3) and the Odometry Model (Sec. 5.4).

Run ./slam in --action-only mode. Run lcm-logplayer-gui
check/uncheck the necessary channels for only running the action model.
Log the data using lcm-logger <name_logfile.log>

DELIVERABLES:

- Provide logs showing the growth of the distribution for:
  - drive_square_10mx10m_5cm.log
  - obstacle_slam_10mx10m_5cm.log
  - Run3_straight_line_calm/straight_line_calm_6_6_2.log
- Provide a final png showing the particle distribution you obtained at the end of each log.
- Write a paragraph or two description including the following:
  - Specify what type of action model you're using.
  - Provide all noise constants that you're using.
  - Comment on the your action model:
    - Does the distribution grow as expected for the particles?
    - Do you think the action model constants will differ drastically from one mbot to another?

1.2.2 MCL - Sensor Model & Particle Filter:

Implement a sensor model that calculates the likelihood of a pose given a laser scan and an occupancy grid map. The skeleton of the sensor model can be found in `slam/sensor_model.h/.cpp`.

Refer to Chapter 6 of Probabilistic Robotics for a discussion of common sensor models. You can base your implementation on the pseudo-code in this chapter.

Finish implementing the particle filter contained in `slam/particle_filter.h/.cpp`.

Refer to Chapter 4 of Probabilistic Robotics for help in implementing your particle filter.

Hint: In case of a slow performance of your sensor model, consider increasing the ray stride in the MovinLaserScan call.

Run ./slam in --localization-only <map_name.map> for each of the runs listed below.
Run lcm-logplayer-gui and check/uncheck the necessary channels.

Log the data using lcm-logger <name_logfile.log>

DELIVERABLES:

- Provide localization logs for:
  - drive_square_10mx10m_5cm.log
  - obstacle_slam_10mx10m_5cm.log
  - Run3_straight_line_calm/straight_line_calm_6_6_2.log
- Provide a final png showing the particle distribution you obtained at the end of each run.
- Write a paragraph or two description including the following:
  - Specify what type of sensor model you're using, and how you are weighting and resampling.
  - Provide all noise constants that you're using.
  - Is the estimated pose closer to the true pose with your localized pose estimate compared to that from odometry?
  - Do the particles remain in a tight region as the robot moves?
  - Do the particles spread more aggressively with a certain motion type? (for example rotation)

1.3 Simultaneous Localization and Mapping (SLAM):

You have now implemented mapping using known poses and localization using a known map. You can now run the slam program in SLAM mode, which uses the following simple SLAM algorithm:

- Use the first laser scan received to construct an initial map.
- For subsequent laser scans:
  - Localize using the map from the previous iteration.
  - Update the map using the pose estimate from your localization.

NOTE: The above logic is already implemented for you. Your goal for this task is to make sure your localization and mapping are accurate enough to provide a stable estimate of the robot pose and map. You will need good SLAM performance to succeed at the robot navigation task.

Run ./slam with no arguments for each of the logs listed below
Run lcm-logplayer-gui and check/uncheck the necessary channels.
Log the data with lcm-logger <name_logfile.log>

DELIVERABLES:

- Provide full slam logs for:
  - obstacle_slam_10mx10m_5cm.log
  - Run1_maze_lowres/Maze_lowres_6_6_2.log
  - Run2_maze_hires/Maze_hires_6_6_2.log
- Provide a final png showing the map obtained at the end of each run.
- Write a paragraph or two description including the following:
  - Did you obtain similar/close performance for obstacle_slam_10mx10m_5cm.log compared to the previous two runs? Why or why not?

    - Did the change of map resolution affect your computation time? Did it improve/harm your slam results?
    - Did you have to change your mapping log odds to improve on performance? If so, report them.
- [Bonus] Provide a log file of a successful run on the Maze_hard log. The bot was driven very aggressively using a dsm controller for this run.